National University – Clark
Bachelor of Science in Information Technology
A.Y. 2026-2027
1st Term

Iskolaria: A Containerized Scholarship Application System with Automated Docker and Jenkins CI/CD Pipeline

A **Project Proposal**
Presented in Partial Fulfillment of the Final Requirements for
Systems Integration and Architecture

Presented by: **Team Ka Ba?**

Cabral, Bea Mei P.
David, Nigel James L.
Serrano, Bianca Gabrielle R.
Soriano, Den Mark C.

ITE302

Submitted to:

Mr. Harwin C. Mendoza

October 2026

I. Project Overview and Team Composition

Item	Details
Group name	Team Ka Ba?
Project title	Iskolaria: A Containerized Scholarship Application System with Automated Docker and Jenkins CI/CD Pipeline
Short Description	Iskolaria is a web-based scholarship management system that allows students to browse scholarships, check eligibility, upload required documents, and track their applications. Scholarship administrators can manage scholarship programs, review applications, verify documents, and update application statuses. The system uses Docker to run its services in separate containers and PostgreSQL to store persistent data. It also implements a Jenkins CI/CD pipeline to automate testing, building, deployment, and smoke testing whenever new code changes are detected.
Repository URL	https://github.com/gxbiyel/Iskolaria
Project Lead / Scrum Master	Bianca Gabrielle R. Serrano
DevOps / CI-CD Engineer	Nigel James L. David
Infrastructure Engineer	Bea Mei P. Cabral
Backend and Database Engineer	Bianca Gabrielle R. Serrano
Frontend, QA, and Documentation Lead	Den Mark C. Soriano

II. System Architecture and Technology Stack

Module / Container	Technology	Purpose	Justification
proxy	Nginx 1.27 Alpine	Single public entry point (port 8080 → 80). Serves pages through the frontend and routes /api/auth/,	Lightweight, gives one public entry point, and routes by path simply. The config is copied into the image, which

		/api/scholarships/, /api/applications/, and /api/uploads/ to the matching service.	avoids bind-mount problems when Jenkins deploys.
frontend	Static HTML/CSS/JS on Nginx	Six pages for students and admins: index, register, login, apply, my, admin. api.js holds shared session, API, and navbar code; build.js shows the build number in the footer.	No build step, so the image stays small and starts quickly. build.js can be generated during the Jenkins image build.
auth-service	Python 3.12, FastAPI	Registration (student number checked against the students table), login with JWT, /me, and health check. Bcrypt hashing, 1-hour tokens, admin account created at startup.	FastAPI gives typed request validation with very little code. JWT (HS256) lets services verify identity and role with a shared secret without querying the database each time, and bcrypt stores passwords only as secure hashes.
scholarship-api	Python 3.12, FastAPI	Manages scholarships (requirements, deadlines, slots) and runs the eligibility check (GWA, family income, year level, deadline, slots), returning every failure reason.	Same FastAPI benefits (validation, little code). Supports the modular split of the scholarship catalog from other concerns.
application-api	Node.js 20, Express	Submits and tracks applications, issues reference numbers, handles cancellation and the admin review workflow, serves notification history and the summary report, and queues notification jobs.	Using a second language shows that containers let different runtimes run side by side. Express suits asynchronous HTTP calls and Redis operations.
upload-service	Python 3.12, FastAPI	Accepts PDF/JPG/PNG up to 2 MB, verifies the real	Isolates file handling as its own module.

		file type from signature bytes, stores files under random names in the uploads-data volume, and lets admins verify or flag documents.	FastAPI validation fits the type and size checks, and the named volume keeps files across redeployments.
worker	Python 3.12	Background processor with no public endpoints. Takes jobs from Redis, saves notifications to PostgreSQL, retries when the DB is down, moves bad jobs to a dead-letter list, and recovers unfinished jobs after a crash.	Separates notification processing from the user's request path, so responses return right away. Redis lists with BLMOVE make the queue reliable.
db	PostgreSQL 16 Alpine	Stores students, users, scholarships, applications, documents, and notifications. Reachable only by backend services. Schema and seed data come from init.sql.	Chosen for relational integrity, UNIQUE constraints (e.g., no duplicate application per student and scholarship), and safe atomic transactions. The pg-data named volume keeps data across redeployments.
redis	Redis 7 Alpine	Queue between application-api and worker, using three lists: notifications, notifications:processing, and notifications:dead.	Decouples notifications from the request path. BLMOVE gives a reliable queue, and failed jobs can go to a dead-letter list.
Jenkins (separate compose)	Jenkins LTS (JDK 17) + Docker CLI	Runs the CI/CD pipeline (Checkout, Test, Build Images, Deploy, Smoke Test), tags images by build number, keeps secrets in Jenkins Credentials, and reports results to Discord.	A declarative Jenkinsfile keeps the pipeline under version control with the code. A separate compose project means redeploying the app never restarts Jenkins, and Poll SCM works even when GitHub's webhook can't reach a local Jenkins.

III. CI/CD Pipeline Design and Implementation Plan

Pipeline Plan	Details
Git hosting	GitHub. The repository holds both the application code and the Jenkinsfile (declarative pipeline, branch main, job set to "Pipeline script from SCM").
Trigger (webhook / polling)	**Both are declared.** Poll SCM (* * * * *) checks the repository every minute and starts a build when it finds a new commit. A GitHub webhook trigger (githubPush) is also declared. Polling is the one the deployment relies on, because GitHub can't reach a Jenkins running locally. Triggers declared in a Jenkinsfile only register after the pipeline has run once, so the first build is started manually and later builds start from pushes.
Planned stages	1. **Checkout**: pulls the exact commit that started the build. 2. **Test**: runs each backend module's tests inside docker build --target test. 3. **Build Images**: docker compose build, with images tagged by build number (iskolaria/<service>:<TAG>). 4. **Deploy**: copies the .env from the Jenkins credential iskolaria-env, then runs docker compose up -d --no-build --remove-orphans. Named volumes are untouched. 5. **Smoke Test**: requests the health endpoints of the auth, scholarship, application, and upload services plus the home page through the proxy, every 5 seconds for up to 60 seconds. Pipeline options: no concurrent builds, last 15 builds kept, 30-minute timeout. A failure before Deploy leaves the running app on the previous version. A Smoke Test failure is reported and recovered by redeploying the last working build.
Tests to be run	Jest for application-api and Pytest for the Python services: • **application-api**: allowed and prohibited status transitions (review can't be skipped, an approved

	application can only be removed, removed is final), cancellation rules, missing-document detection • **auth-service**: student number normalization, bcrypt hashing and verification, JWT creation and decoding, rejection of expired or wrongly signed tokens • **scholarship-api**: eligibility (GWA, family income ceiling, year level, deadline, slots) and input validation (GWA outside 1.00 to 5.00, negative income) • **upload-service**: file type from signature bytes, rejection of executables and renamed text files, 2 MB limit, content-type to extension matching • **worker**: a message for every status, fallback for unknown statuses, job validation (invalid JSON, missing fields, unknown statuses, non-integer IDs) • **Smoke test** (post-deploy, as above)
Planned bonus features (optional)	• **Discord notifications** of every build result, with the webhook URL kept in Jenkins Credentials (discord-webhook) • **Rollback by build-number tags**, since tagged images are never pruned and the footer shows the running build number • **Secrets handling**: .env injected from a Secret file credential and deleted in the post section after every run • **Automated smoke test with retries** after deployment • **Test-in-Docker approach**: tests run in a dedicated build target, so no language dependencies are needed on the Jenkins host • **Failure demo**: a commit that lets "submitted" jump straight to "approved" breaks the "cannot skip review" test and blocks deployment

IV. System Architecture Diagram

